

Adding Microsoft Clarity to a PHP site

Clarity records real user sessions — replays, click and scroll heatmaps, rage clicks and JavaScript errors. This is everything needed to wire it into a PHP/MySQL application, including how to tie a recording back to a logged-in user.

CLARITY PROJECT ID	FILES TO CHANGE	ESTIMATED TIME	SERVER REQUIREMENT
y0lgk0duzw	2	~30 minutes	None

Before you start

Clarity is entirely client-side — a small asynchronous script tag. There is nothing to install with Composer, no PHP extension, no database table, and no change to your server configuration. It will not slow page loads; the script is loaded `async` and does not block rendering.

The project ID above is the one identifier that matters. Every page that loads the tag with that ID reports into the same Clarity dashboard. If this PHP application is a *different* site from the one already tracked, create a separate project at clarity.microsoft.com and use its own ID instead — mixing two unrelated sites into one project makes every metric a blend of both.

STEP 01 — THE TAG

Create a single include file

Put the tag in one file. Everything else in this guide edits that file, and never the individual pages.

clarity.php

```
<?php
// Microsoft Clarity – behaviour analytics.
// Included once, from the shared page header. Never per-page.

$clarityId = defined('CLARITY_ID') ? CLARITY_ID : 'y0lgk0duzw';

if ($clarityId): ?>
<script type="text/javascript">
    (function(c,l,a,r,i,t,y){
        c[a]=c[a]||function(){(c[a].q=c[a].q||[]).push(arguments)};
        t=l.createElement(r);t.async=1;t.src="https://www.clarity.ms/tag/"+i;
        y=l.getElementsByTagName(r)[0];y.parentNode.insertBefore(t,y);
    })(window, document, "clarity", "script", <?= json_encode($clarityId) ?>);
</script>
<?php endif; ?>
```

Reading the ID from a constant means it can be switched per environment. Leave `CLARITY_ID` undefined — or set it to an empty string — on staging and local machines, and the tag is never emitted, so test clicks never pollute production data.

STEP 02 – PLACEMENT

Include it from the shared header, once

The tag belongs in `<head>`, in whichever file every page already includes — `header.php`, a layout template, the theme header.

header.php

```
<head>
  <meta charset="utf-8">
  <title><?= htmlspecialchars($pageTitle) ?></title>

  <?php include __DIR__ . '/clarity.php'; ?>
</head>
```

INCLUDE IT EXACTLY ONCE

Pasting the snippet into individual pages as well will load the tag twice on those pages. The result is duplicated events, inflated session counts and unreliable heatmaps — and it is difficult to spot after the fact, because the dashboard still looks plausible. One include, in the shared header.

If there is no shared header

Some older applications have no template at all. Creating even a two-line shared head file is the right fix and is usually an hour's work. As a stopgap, `auto_prepend_file` runs a file before every request without touching a single page:

.htaccess

```
php_value auto_prepend_file /var/www/html/clarity-boot.php
```

Treat this as temporary. It cannot target `<head>` specifically without output buffering, and it also fires for AJAX endpoints, cron scripts and file downloads, where injecting a script tag will corrupt the response.

STEP 03 — IDENTITY

Tie recordings to the logged-in user

Without this, every recording is anonymous and you cannot answer "show me what this customer did before they called us". PHP has the advantage here: the user is already known server-side, in the session.

Add an escaping helper — this is not optional, see the warning below — and then the identify block.

helpers.php – include before clarity.php

```
<?php
/**
 * Safely embed a PHP value inside a <script> block.
 * JSON_HEX_TAG is what stops a value containing "</script>"
 * from breaking out of the script element.
 */
function js($value) {
    return json_encode(
        $value,
        JSON_HEX_TAG | JSON_HEX_AMP | JSON_HEX_APOS | JSON_HEX_QUOT
    );
}
```

clarity.php – append below the tag

```
<?php if (!empty($_SESSION['user_phone'])): ?>
<script>
    // Clarity hashes the ID before storing it. The last argument is the
    // "friendly name" shown in the recordings list.
    clarity("identify",
        <?= js($_SESSION['user_phone']) ?>,
        undefined,
        undefined,
        <?= js($_SESSION['user_name'] ?? $_SESSION['user_phone']) ?>
    );

    // Custom tags – each one becomes a filter in the Clarity dashboard.
    clarity("set", "user_phone", <?= js($_SESSION['user_phone']) ?>);
    clarity("set", "user_role", <?= js($_SESSION['role'] ?? 'user') ?>);
    clarity("set", "city", <?= js($_SESSION['city'] ?? '') ?>);
</script>
<?php endif; ?>
```

SECURITY – DO NOT SKIP THE ESCAPING

Never write a database or session value directly into a `<script>` block. A name stored in MySQL containing `</script>` closes the element early and everything after it is parsed as HTML — a stored cross-site scripting hole. Analytics snippets are a classic place for this bug, precisely because the value feels "internal". Always route it through `js()`.

SESSIONS

`$_SESSION` is only populated after `session_start()` has run. If the header is included before the session is opened, these values will be silently empty and every recording will be anonymous. Check the order.

Once deployed, these tags appear in the dashboard under **Filters** → **Custom tags**. Filtering by one applies to the whole dashboard, not just the recordings list — so selecting a single `user_phone` recalculates every metric for that one person.

STEP 04 – PRIVACY

Mask the data you don't want recorded

Session replays capture what is on screen. Anything rendered from the database — customer phone numbers, email addresses, payment details — ends up in the recording unless it is masked.

Set the baseline in the dashboard under **Settings** → **Masking** (Strict is the safe default for any page showing customer records), then override per element where needed. The attributes work regardless of what generated the HTML:

Any template file

```
<!-- Never recorded, in any masking mode -->
<td data-clarity-mask="true"><?= htmlspecialchars($row['phone']) ?></td>
<td data-clarity-mask="true"><?= htmlspecialchars($row['email']) ?></td>

<!-- Always recorded, even in Strict mode -->
<div data-clarity-unmask="true"><?= htmlspecialchars($errorMessage) ?></div>
```

Password and OTP inputs are never recorded by Clarity, regardless of settings. Unmasking error messages is worth doing deliberately — being able to read the actual failure text in a replay is often the whole reason you opened it.

VERIFICATION

Confirm it is actually working

Do not wait on the dashboard to tell you. Load any page, open DevTools (**F12**), and check two things:

WHERE	WHAT TO LOOK FOR	MEANS
Network tab, filter <code>clarity</code>	A request to <code>clarity.ms/tag/...</code> returning 200	The tag downloaded
Network tab	<code>POST v.clarity.ms/collect</code> returning 204	Session data is being accepted
Console	Type <code>clarity</code> — prints a function	The API initialised
Console	<code>clarity("event", "manual_test")</code>	End-to-end proof; findable in the dashboard

Recordings appear within a few minutes. Heatmaps need more traffic before they render anything meaningful — give them a day of real usage before judging the result.

Two things that will bite you

FULL-PAGE CACHING

If the site sits behind Varnish, a static HTML cache or a plugin like WP Super Cache, the identify block gets cached along with the *first* visitor's identity and then served to everyone else — so every session is attributed to one person. Either exclude that fragment from caching (an ESI include or a cache hole), or set the identity from a small client-side call that runs after the cached HTML is delivered.

AD BLOCKERS

uBlock Origin, Brave's shields and some corporate DNS filters block `clarity.ms` outright. A meaningful share of sessions will simply never be recorded. This is normal and not a fault in the integration — do not treat Clarity's session count as a traffic figure.

Checklist

☐ `clarity.php` created, project ID read from a constant

☐ Included once, inside `<head>`, from the shared header

☐ No copy of the snippet left in any individual page

☐ `js()` helper added and used for every injected value

☐ `session_start()` confirmed to run before the header

☐ Masking mode set in the dashboard; PII columns tagged `data-clarity-mask`

☐ Verified in DevTools: tag 200, `collect` 204

☐ Staging and local environments leave the ID unset

Reference: learn.microsoft.com/clarify · API methods used here — `identify`, `set`, `event` — are documented under "Clarity API".